Phase 6 Notes: Dividing Functions with Linear Work

1. Current Progress Analysis

You are now at **Phase 6**.

Before this phase, you already learned how to read recursive code and write recurrence relations.

What you already know

Phase	Main Idea	Example Recurrence	Final Result
Phase 1	Turn recursive code into a recurrence relation	`T(n) = T(n - 1) + 1`	`O(n)`
Phase 2	Solve one-call decreasing recurrences	`T(n) = T(n - 1) + 1`	`O(n)`
Phase 3	Handle multiple decreasing calls	`T(n) = 2T(n - 1) + 1`	`O(2^n)`
Phase 4	Use the decreasing master theorem	`T(n) = aT(n - b) + f(n)`	Depends on `a`
Phase 5	Divide the input by 2 with constant work	`T(n) = T(n / 2) + 1`	`O(log n)`

Important idea from Phase 5

In Phase 5, the input was divided by 2 each time.

Example:

```
8 -> 4 -> 2 -> 1
```

This takes only a few steps.

That is why:

```
T(n) = T(n / 2) + 1
```

gives:

```
O(log n)
```

The reason is simple:

> Repeatedly cutting `n` in half creates `log n` levels.

---

2. Main Goal of Phase 6

In Phase 6, we still divide the input by 2.

But now, each call does more work before making the recursive call.

In Phase 5, each call did only constant work:

```
cout << n;      // O(1)
algorithm(n / 2);
```

So the recurrence was:

```
T(n) = T(n / 2) + 1
```

In Phase 6, each call runs a loop from `1` to `n`:

```
for (int i = 1; i <= n; i++) {
    cout << i << " ";
}
algorithm(n / 2);
```

The loop runs `n` times.

So the current call costs:

```
n
```

Therefore, the recurrence becomes:

```
T(n) = T(n / 2) + n
```

The final time complexity is:

```
O(n)
```

Main lesson of Phase 6

> One recursive call on half the input does not always mean `O(log n)`.
>
> You must also count the work done at each level.

---

3. What Is a Dividing Function with Linear Work?

A **dividing function** is a recursive function where the input becomes smaller by division.

Example:

```
algorithm(n / 2);
```

This means the next recursive call receives half of the current input.

A **linear-work dividing function** means that before or after the recursive call, the function does work proportional to `n`.

The most common example is a loop from `1` to `n`:

```
for (int i = 1; i <= n; i++) {
    cout << i << " ";
}
```

This loop runs `n` times, so it costs:

```
O(n)
```

So Phase 6 combines two ideas:

1. The input is divided by 2.
2. Each level does linear work.

---

4. Main Code Example

```cpp
#include <iostream>
using namespace std;

void algorithm(int n) {
    if (n > 1) {
        for (int i = 1; i <= n; i++) {
            cout << i << " ";
        }

        cout << "| ";

        algorithm(n / 2);
    }
}

int main() {
    int n = 8;
    cout << "Output: ";
    algorithm(n);
    cout << endl;
    return 0;
}
```

---

5. What the Code Does

The function is:

```cpp
void algorithm(int n) {
    if (n > 1) {
        for (int i = 1; i <= n; i++) {
            cout << i << " ";
        }

        cout << "| ";

        algorithm(n / 2);
    }
}
```

It does three main things:

1. It checks whether `n > 1`.
2. If true, it runs a loop from `1` to `n`.
3. Then it calls itself with `n / 2`.

The function stops when `n` becomes `1` or smaller.

---

6. Breaking the Code Into Parts

Part 1: Base condition

```
if (n > 1)
```

The function continues only while `n > 1`.

When `n = 1`, the condition becomes false:

```
1 > 1 is false
```

So the function stops.

The base case is:

```
T(1) = 1
```

This means the stopping case takes constant time.

Part 2: Current work

```
for (int i = 1; i <= n; i++) {
    cout << i << " ";
}
```

This loop runs from `1` to `n`.

So it runs `n` times.

Therefore, the current work is:

```
n
```

or:

```
O(n)
```

This is why Phase 6 is called **linear work**.

Part 3: Recursive call

```
algorithm(n / 2);
```

There is only one recursive call.

The input changes from:

```
n
```

to:

```
n / 2
```

So the recursive part is:

```
T(n / 2)
```

---

7. Writing the Recurrence Relation

The total time is:

```
current work + recursive call time
```

The current work is:

```
n
```

The recursive call time is:

```
T(n / 2)
```

So the recurrence is:

```
T(n) = T(n / 2) + n
```

Base case:

```
T(1) = 1
```

Full recurrence:

```
T(n) = T(n / 2) + n, for n > 1
T(1) = 1
```

---

8. Meaning of Each Part of the Recurrence

The recurrence is:

```
T(n) = T(n / 2) + n
```

Part	Meaning
`T(n)`	Time taken for input size `n`
`T(n / 2)`	Time taken by the recursive call on half the input
`+ n`	Linear work done in the current call
`T(1) = 1`	Base case takes constant time

In simple words:

> The function does `n` work now, then solves the same problem again for half the input.

---

9. General Division Recurrence Form

Dividing recurrences usually follow this form:

```
T(n) = aT(n / b) + f(n)
```

Where:

Symbol	Meaning
`a`	Number of recursive calls
`b`	Division factor
`f(n)`	Extra work done outside recursion

For Phase 6:

```
T(n) = T(n / 2) + n
```

So:

Symbol	Value	Reason
`a`	`1`	There is one recursive call
`b`	`2`	The input is divided by 2
`f(n)`	`n`	The loop runs `n` times

So the recurrence can also be written as:

```
T(n) = 1T(n / 2) + n
```

Simplified:

```
T(n) = T(n / 2) + n
```

---

10. Dry Run of `algorithm(8)`

Call:

```
algorithm(8);
```

Step-by-step table

Call	Condition	Work Done	Next Call
`algorithm(8)`	`8 > 1` is true	prints `1 2 3 4 5 6 7 8`	`algorithm(4)`
`algorithm(4)`	`4 > 1` is true	prints `1 2 3 4`	`algorithm(2)`
`algorithm(2)`	`2 > 1` is true	prints `1 2`	`algorithm(1)`
`algorithm(1)`	`1 > 1` is false	stops	no call

Output pattern

```
1 2 3 4 5 6 7 8 | 1 2 3 4 | 1 2 |
```

Total print operations

```
8 + 4 + 2 = 14
```

For `n = 8`, the function does 14 units of print work.

---

11. Work Pattern for Larger `n`

For input `n`, the work by level looks like this:

Level	Input Size	Work at This Level
Level 0	`n`	`n`
Level 1	`n / 2`	`n / 2`
Level 2	`n / 4`	`n / 4`
Level 3	`n / 8`	`n / 8`
...	...	...
Last level	`1`	stops

So the total work is:

```
n + n/2 + n/4 + n/8 + ...
```

This is a geometric series.

---

12. What Is a Geometric Series Here?

The work forms this series:

```
n + n/2 + n/4 + n/8 + ...
```

Each term is half of the previous term.

Example with `n = 64`:

```
64 + 32 + 16 + 8 + 4 + 2
```

Each level does less work than the level before it.

This is different from `n log n`, where each level would cost about `n`.

Here, the cost decreases:

```
n, n/2, n/4, n/8, ...
```

---

13. Why the Sum Becomes `O(n)`

The total work is:

```
n + n/2 + n/4 + n/8 + ...
```

Factor out `n`:

```
n(1 + 1/2 + 1/4 + 1/8 + ...)
```

The inside part is:

```
1 + 1/2 + 1/4 + 1/8 + ...
```

This sum approaches `2`.

So:

```
n(1 + 1/2 + 1/4 + 1/8 + ...) ≈ n(2)
```

That gives:

```
2n
```

In Big O notation, constants do not matter.

So:

```
O(2n) = O(n)
```

Final time complexity:

```
O(n)
```

---

14. Solving the Recurrence by Successive Substitution

We solve:

```
T(n) = T(n / 2) + n
```

Step 1: Start with the recurrence

```
T(n) = T(n / 2) + n
```

Step 2: Substitute `T(n / 2)`

Using the same recurrence rule:

```
T(n / 2) = T(n / 4) + n / 2
```

Substitute into the original recurrence:

```
T(n) = [T(n / 4) + n / 2] + n
```

So:

```
T(n) = T(n / 4) + n / 2 + n
```

Step 3: Substitute again

Using the same rule:

```
T(n / 4) = T(n / 8) + n / 4
```

Substitute:

```
T(n) = [T(n / 8) + n / 4] + n / 2 + n
```

So:

```
T(n) = T(n / 8) + n / 4 + n / 2 + n
```

Step 4: See the pattern

After 1 step:

```
T(n) = T(n / 2) + n
```

After 2 steps:

```
T(n) = T(n / 4) + n / 2 + n
```

After 3 steps:

```
T(n) = T(n / 8) + n / 4 + n / 2 + n
```

Since:

```
2 = 2^1
4 = 2^2
8 = 2^3
```

After `k` steps:

```
T(n) = T(n / 2^k) + n + n/2 + n/4 + ... + n/2^(k - 1)
```

This is the general pattern.

---

15. Reaching the Base Case

The base case is:

```
T(1) = 1
```

So we want the input inside `T(...)` to become `1`.

The pattern is:

```
T(n) = T(n / 2^k) + n + n/2 + n/4 + ...
```

We need:

```
n / 2^k = 1
```

Now solve this equation.

Multiply both sides by `2^k`:

```
n = 2^k
```

Take logarithm base 2:

```
k = log₂(n)
```

So the recursion reaches the base case after about:

```
log₂(n)
```

levels.

---

16. Substitute Back

The pattern is:

```
T(n) = T(n / 2^k) + n + n/2 + n/4 + ... + n/2^(k - 1)
```

We found:

```
k = log₂(n)
```

When `k = log₂(n)`, this part becomes the base case:

```
T(n / 2^k) = T(1)
```

So:

```
T(n) = T(1) + n + n/2 + n/4 + ...
```

Since:

```
T(1) = 1
```

we get:

```
T(n) = 1 + n + n/2 + n/4 + ...
```

The series:

```
n + n/2 + n/4 + ...
```

is about:

```
2n
```

So:

```
T(n) = 1 + 2n
```

In Big O:

```
T(n) = O(n)
```

---

17. Full Solution in One Place

Start:

```
T(n) = T(n / 2) + n
```

Substitute:

```
T(n) = T(n / 4) + n/2 + n
```

Substitute again:

```
T(n) = T(n / 8) + n/4 + n/2 + n
```

Pattern:

```
T(n) = T(n / 2^k) + n + n/2 + n/4 + ... + n/2^(k - 1)
```

Use the base case:

```
n / 2^k = 1
```

Solve:

```
n = 2^k
k = log₂(n)
```

The work sum is:

```
n + n/2 + n/4 + n/8 + ...
```

Factor out `n`:

```
n(1 + 1/2 + 1/4 + 1/8 + ...)
```

The parentheses sum to about `2`:

```
n(2) = 2n
```

Final:

```
T(n) = O(n)
```

---

18. Beginner-Friendly Meaning of the Solution

The recurrence:

```
T(n) = T(n / 2) + n
```

means:

> Do `n` units of work now, then solve the same problem for half the input.

The calls go like this:

```
n
n / 2
n / 4
n / 8
n / 16
...
1
```

But the work also gets smaller:

```
n + n/2 + n/4 + n/8 + ...
```

The total does not grow like `n log n` because each level is not `n` work.

The levels get cheaper and cheaper.

So the total is only about:

```
2n
```

Therefore:

```
O(n)
```

---

19. Why the Answer Is Not `O(log n)`

A common beginner mistake is to say:

> The input is divided by 2, so the answer must be `O(log n)`.

This is not always true.

Dividing by 2 tells us the number of levels:

```
O(log n) levels
```

But we must also ask:

> How much work happens at each level?

In Phase 5:

```
T(n) = T(n / 2) + 1
```

Each level costs `1`.

So:

```
1 + 1 + 1 + ... for log n levels = O(log n)
```

In Phase 6:

```
T(n) = T(n / 2) + n
```

The levels cost:

```
n + n/2 + n/4 + ...
```

This sum is:

```
O(n)
```

So the answer is not `O(log n)`.

It is:

```
O(n)
```

---

20. Why the Answer Is Not `O(n log n)`

Another common mistake is to say:

> There are `log n` levels and a loop of size `n`, so the answer is `O(n log n)`.

This is also wrong for Phase 6.

The first level costs `n`, but the next level does not also cost `n`.

The cost shrinks:

```
n
n/2
n/4
n/8
...
```

So we do not have:

```
n + n + n + n + ...
```

That would be `n log n`.

Instead, we have:

```
n + n/2 + n/4 + n/8 + ...
```

That is about `2n`.

So the final answer is:

```
O(n)
```

---

21. Time Complexity

The time complexity is:

```
O(n)
```

Why?

Because the total work is:

```
n + n/2 + n/4 + n/8 + ...
```

This series approaches:

```
2n
```

And Big O ignores constants:

```
O(2n) = O(n)
```

So:

```
Time complexity = O(n)
```

---

22. Stack Space Complexity

The recursion stack space is:

```
O(log n)
```

Why?

Because there is only one recursive call each time:

```
algorithm(n / 2);
```

The calls form one chain:

```
algorithm(n)
algorithm(n / 2)
algorithm(n / 4)
algorithm(n / 8)
...
algorithm(1)
```

The chain length is about:

```
log n
```

So the stack space is:

```
O(log n)
```

---

23. Time Complexity vs Stack Space

For Phase 6:

Recurrence	Time Complexity	Stack Space
`T(n) = T(n / 2) + n`	`O(n)`	`O(log n)`

Why time is `O(n)`

The work is:

```
n + n/2 + n/4 + ... = O(n)
```

Why space is `O(log n)`

The recursive calls form one chain of length:

```
log n
```

So stack space is:

```
O(log n)
```

---

24. Comparison Between Phase 5 and Phase 6

Feature	Phase 5	Phase 6
Recurrence	`T(n) = T(n / 2) + 1`	`T(n) = T(n / 2) + n`
Recursive calls	1	1
Input change	`n / 2`	`n / 2`
Work per level	constant	linear, but shrinking
Work series	`1 + 1 + 1 + ...`	`n + n/2 + n/4 + ...`

Feature	Phase 5	Phase 6
Number of levels	`log n`	`log n`
Time complexity	`O(log n)`	`O(n)`
Stack space	`O(log n)`	`O(log n)`

Key difference

Both phases have `log n` recursive levels.

But Phase 5 does constant work at each level.

Phase 6 does linear work at the first level, then smaller linear work at each later level.

That is why Phase 6 is larger than Phase 5.

---

25. Important Terms in Phase 6

Dividing function

A recursive function where the input becomes smaller by division.

Example:

```
algorithm(n / 2);
```

Division recurrence

A recurrence where the recursive part contains division.

Example:

```
T(n) = T(n / 2) + n
```

Linear work

Work that grows directly with `n`.

Example:

```
for (int i = 1; i <= n; i++) {
    cout << i << " ";
}
```

This loop runs `n` times, so it is linear work.

Base case

The condition where recursion stops.

In this phase:

```
if (n > 1)
```

The function stops when:

```
n = 1
```

So:

```
T(1) = 1
```

Geometric series

A series where each term is multiplied by the same ratio.

In Phase 6:

```
n + n/2 + n/4 + n/8 + ...
```

Each term is half of the previous term.

The sum is about:

```
2n
```

So the final result is:

```
O(n)
```

Successive substitution

A method where we repeatedly replace the smaller recurrence until we find a pattern.

Example:

```
T(n) = T(n / 2) + n
T(n) = T(n / 4) + n/2 + n
T(n) = T(n / 8) + n/4 + n/2 + n
```

Pattern:

```
T(n) = T(n / 2^k) + n + n/2 + n/4 + ...
```

---

26. Complete Coding Program

```cpp
#include <iostream>
using namespace std;

// This function does linear work, then calls itself with n / 2.
void algorithm(int n) {
    if (n > 1) {
        // Linear work at this level: O(n)
        for (int i = 1; i <= n; i++) {
            cout << i << " ";
        }

        cout << "| ";

        // One recursive call on half the input
        algorithm(n / 2);
    }
```

```
}

int main() {
    int n = 8;
    cout << "Output: ";
    algorithm(n);
    cout << endl;
    return 0;
}
```

Output for `n = 8`

```
Output: 1 2 3 4 5 6 7 8 | 1 2 3 4 | 1 2 |
```

Recurrence

```
T(n) = T(n / 2) + n
T(1) = 1
```

Time complexity

```
O(n)
```

Stack space

```
O(log n)
```

---

27. Common Beginner Mistakes

Mistake 1: Assuming every dividing function is `O(log n)`

Wrong idea:

```
T(n) = T(n / 2) + n gives O(log n)
```

Correct idea:

```
T(n) = T(n / 2) + n gives O(n)
```

Why?

Because the function does not only divide the input.

It also runs a loop.

The loop work creates this sum:

```
n + n/2 + n/4 + ...
```

That sum is `O(n)`.

Mistake 2: Thinking the answer is `O(n log n)`

Wrong idea:

```
There are log n levels and a loop, so it must be O(n log n).
```

Correct idea:

The loop size is not `n` at every level.

The loop size shrinks:

```
n, n/2, n/4, n/8, ...
```

So the total is:

```
O(n)
```

Mistake 3: Adding the series incorrectly

Wrong idea:

```
n + n/2 + n/4 + ... = n log n
```

Correct idea:

```
n + n/2 + n/4 + ... ≈ 2n
```

So:

```
O(2n) = O(n)
```

Mistake 4: Forgetting that Big O ignores constants

The sum is about:

```
2n
```

But Big O ignores the constant `2`.

So:

```
O(2n) = O(n)
```

Mistake 5: Ignoring stack space

The time is:

```
O(n)
```

But the stack space is not also `O(n)`.

There is only one recursive call each time, and the input is divided by 2.

So the stack depth is:

```
O(log n)
```

28. Step-by-Step Checklist for Phase 6 Problems

When you see a recursive function like this, follow these steps.

Step 1: Find the base case

Look for the stopping condition.

Example:

```
if (n > 1)
```

The function stops when:

```
n = 1
```

So:

```
T(1) = 1
```

Step 2: Count recursive calls

Example:

```
algorithm(n / 2);
```

There is one recursive call.

So:

```
a = 1
```

Step 3: Track how the input changes

The input changes from:

```
n
```

to:

```
n / 2
```

So the recursive part is:

```
T(n / 2)
```

Step 4: Count extra work

Example:

```
for (int i = 1; i <= n; i++) {
    cout << i << " ";
}
```

The loop runs `n` times.

So the extra work is:

```
+n
```

Step 5: Write the recurrence

```
T(n) = T(n / 2) + n
T(1) = 1
```

Step 6: Substitute repeatedly

```
T(n) = T(n / 2) + n
T(n) = T(n / 4) + n/2 + n
T(n) = T(n / 8) + n/4 + n/2 + n
```

Step 7: Find the pattern

```
T(n) = T(n / 2^k) + n + n/2 + n/4 + ... + n/2^(k - 1)
```

Step 8: Use the base case

```
n / 2^k = 1
```

Step 9: Solve for `k`

```
n = 2^k
k = log₂(n)
```

Step 10: Add the work series

```
n + n/2 + n/4 + n/8 + ...
```

This is about:

```
2n
```

Step 11: Write the final Big O

```
T(n) = O(n)
```

Stack space:

```
O(log n)
```

---

29. Mini Practice Example 1

Find the recurrence and time complexity.

```
void test(int n) {
    if (n > 1) {
        for (int i = 1; i <= n; i++) {
            cout << "*";
        }
        test(n / 2);
    }
}
```

Solution

Base case:

```
T(1) = 1
```

There is one recursive call:

```
test(n / 2);
```

So the recursive part is:

```
T(n / 2)
```

The loop runs `n` times, so the current work is:

```
+n
```

Recurrence:

```
T(n) = T(n / 2) + n
```

Time complexity:

```
O(n)
```

Stack space:

```
O(log n)
```

---

30. Mini Practice Example 2

Find the recurrence and time complexity.

```
void countHalfWithLoop(int n) {
    if (n > 1) {
        for (int i = 0; i < n; i++) {
            cout << i;
        }
        countHalfWithLoop(n / 2);
    }
}
```

Solution

The function has one recursive call:

```
countHalfWithLoop(n / 2);
```

So the recursive part is:

```
T(n / 2)
```

The loop runs `n` times:

```
for (int i = 0; i < n; i++)
```

So the extra work is:

```
+n
```

Recurrence:

```
T(n) = T(n / 2) + n
T(1) = 1
```

Final time:

```
O(n)
```

Stack space:

```
O(log n)
```

---

31. Simple Way to Remember Phase 6

For this code pattern:

```
function(n) {
    if (n > 1) {
        do n work;
        function(n / 2);
    }
}
```

The recurrence is:

```
T(n) = T(n / 2) + n
```

The time complexity is:

```
O(n)
```

The stack space is:

```
O(log n)
```

Memory trick

```
T(n) = T(n / 2) + 1  -> O(log n)
T(n) = T(n / 2) + n  -> O(n)
```

Another memory trick

```
Constant work at each half-size level -> O(log n)
Shrinking linear work at each half-size level -> O(n)
```

Most important memory sentence

`n + n/2 + n/4 + ...` is about `2n`, so it is `O(n)`.

---

32. Phase 6 Final Summary

In Phase 6, you learned about dividing functions with linear work.

The main code was:

```
void algorithm(int n) {
    if (n > 1) {
        for (int i = 1; i <= n; i++) {
            cout << i << " ";
        }
        algorithm(n / 2);
    }
}
```

The function:

1. Runs a loop from `1` to `n`.
2. Calls itself with `n / 2`.
3. Stops when `n = 1`.

The recurrence is:

```
T(n) = T(n / 2) + n
T(1) = 1
```

Using substitution:

```
T(n) = T(n / 2) + n
T(n) = T(n / 4) + n/2 + n
T(n) = T(n / 8) + n/4 + n/2 + n
```

Pattern:

```
T(n) = T(n / 2^k) + n + n/2 + n/4 + ... + n/2^(k - 1)
```

Base case equation:

```
n / 2^k = 1
```

Solve:

```
k = log₂(n)
```

Work series:

```
n + n/2 + n/4 + n/8 + ...
```

The sum is about:

```
2n
```

Final result:

```
Time complexity: O(n)
Stack space: O(log n)
```

The most important lesson is:

> Dividing the input by 2 gives `log n` levels, but the total time also depends on how much work happens at each level.
>
> In Phase 6, the work is `n + n/2 + n/4 + ...`, so the answer is `O(n)`.

---

33. What You Should Be Able to Do After Phase 6

After Phase 6, you should be able to:

1. Recognize a dividing recursive function with linear work.
2. Understand that `T(n / 2)` is different from `T(n - 1)`.
3. Identify the base case for a dividing function.
4. Count the number of recursive calls.
5. Identify that `algorithm(n / 2)` gives the recursive term `T(n / 2)`.
6. Identify a loop from `1` to `n` as linear work.
7. Write the recurrence `T(n) = T(n / 2) + n`.
8. Solve the recurrence using successive substitution.
9. Find the pattern `T(n) = T(n / 2^k) + n + n/2 + n/4 + ...`.
10. Use the base case equation `n / 2^k = 1`.
11. Understand why `k = log₂(n)`.
12. Understand the geometric series `n + n/2 + n/4 + ...`.
13. Know that this series is about `2n`.
14. Conclude that the time complexity is `O(n)`.
15. Understand why the stack space is `O(log n)`.
16. Avoid the mistake of saying the answer is automatically `O(log n)`.
17. Avoid the mistake of saying the answer is `O(n log n)`.
18. Explain in simple words why the final answer is `O(n)`.

---

34. Key Formulas From Phase 6

Main recurrence:

```
T(n) = T(n / 2) + n
T(1) = 1
```

After substitution:

```
T(n) = T(n / 2^k) + n + n/2 + n/4 + ... + n/2^(k - 1)
```

Base case equation:

```
n / 2^k = 1
```

Solve:

```
n = 2^k
k = log₂(n)
```

Work series:

```
n + n/2 + n/4 + n/8 + ...
```

Series result:

```
≈ 2n
```

Final result:

```
Time: O(n)
Space: O(log n)
```

Most important memory sentence:

```
If one recursive call cuts n in half but each level does linear work, the answer
is O(n), not O(log n).
```